

GPUMODE Lecture Study Notes: Lecture 97

HipKittens

Core Problem the Lecture Addresses

The lecture studies the problem of writing high-performance AI kernels for AMD GPUs, especially kernels that must compete with hand-written assembly while still remaining understandable, reusable, and productive for kernel developers. The specific system discussed is **HipKittens**, a library inspired by ThunderKittens but adapted to the architectural realities of AMD CDNA GPUs.

The central technical problem is:

How do we expose enough low-level control to hit AMD GPU peak performance, without forcing every AI kernel author to write raw assembly?

This matters because modern AI workloads are dominated by long-running training and inference jobs on clusters that may cost hundreds of millions of dollars. A seemingly small kernel inefficiency can translate into enormous wasted compute. The lecture emphasizes that a hardware-efficient algorithm on one GPU generation may degrade badly on another GPU generation unless the kernel implementation is adapted to the new hardware.

A key motivating example is the degradation of FlashAttention-style GPU kernels when moving from one NVIDIA generation to another. The broader lesson is not specific to FlashAttention: **kernel performance is not portable by default**. Hardware changes, tensor core instruction changes, memory hierarchy changes, cache topology changes, and compiler behavior all affect whether an algorithm actually reaches the roofline.

The speaker frames the problem around three categories of performance work:

1. **Explicit memory movement**: moving data correctly between HBM, cache, shared memory, and registers.
2. **Occupancy and scheduling**: arranging compute and memory work so the GPU's execution units are saturated.

3. **Cache reuse:** scheduling work across thread blocks and chiplets so reuse is preserved in non-programmer-managed cache levels.

The main mental model is that AMD GPUs have competitive hardware, but their performance is locked behind software that must understand AMD-specific details: wave size, matrix core layouts, register file partitioning, shared memory behavior, memory instructions, and chiplet-level cache topology.

Required Background

What a Kernel Does

A GPU kernel is a program launched over a large parallel grid. Each instance of work is executed by a thread, and threads are grouped into blocks. On NVIDIA, a group of 32 threads executes as a warp. On AMD, the analogous unit is called a **wave**, and for the CDNA hardware discussed in the lecture it generally contains 64 threads.

A kernel usually performs the following sequence:

1. Load data from global memory, usually HBM.
2. Stage or transform data through caches, shared memory, and registers.
3. Execute arithmetic, often using matrix core instructions for AI workloads.
4. Store results back to global memory.

The basic kernel performance problem is:

$$T_{\text{kernel}} \approx \max(T_{\text{memory}}, T_{\text{compute}}, T_{\text{sync}}, T_{\text{launch}}),$$

where T_{memory} is time spent moving data, T_{compute} is time spent doing arithmetic, T_{sync} is time spent waiting at synchronization points, and T_{launch} is kernel launch and teardown overhead.

GPU Memory Hierarchy

A modern GPU has a hierarchy of storage locations. The closer a location is to the execution units, the smaller and faster it tends to be.

Registers $\rightarrow$ Shared Memory $\rightarrow$ L1 or Local Cache $\rightarrow$ L2 Cache $\rightarrow$ LLC $\rightarrow$ HBM.

The general latency and capacity trade-off is:

Registers: smallest and fastest,

HBM: largest and slowest.

For AI kernels, HBM capacity is essential because model parameters, activations, and KV caches can be hundreds of gigabytes. However, peak compute can only be reached if data is moved from HBM into registers and matrix cores efficiently.

Compute Units, SMs, Waves, and Blocks

On NVIDIA, the main programmable processor unit is called an SM, or streaming multiprocessor. On AMD, the analogous unit is called a CU, or compute unit. The lecture uses the vocabulary:

AMD thread $\rightarrow$ wave $\rightarrow$ thread block $\rightarrow$ CU $\rightarrow$ XCD $\rightarrow$ GPU.

A simplified AMD hierarchy discussed in the lecture is:

64 threads = 1 wave,

32 waves $\approx$ 1 CU,

32 CUs $\approx$ 1 XCD,

8 XCDs $\approx$ 1 GPU.

The exact numbers vary by chip and generation, but this model is useful for understanding why AMD kernels must care about chiplets and cache locality.

Tensor Cores and Matrix Cores

AI workloads increasingly depend on specialized matrix multiply hardware. NVIDIA calls these units tensor cores. AMD uses matrix core instructions, commonly exposed through MFMA-style instructions.

The basic operation is a matrix multiply-accumulate:

$$D \leftarrow AB + C.$$

For a tile-level instruction, this is usually:

$$D_{m \times n} \leftarrow A_{m \times k} B_{k \times n} + C_{m \times n}.$$

The key performance point is:

If a kernel does not use matrix cores, it is usually not using the GPU effectively.

Vector ALUs and transcendental units still matter for operations such as elementwise arithmetic, exponentials, logarithms, reductions, normalization, and softmax, but the peak FLOP rate is dominated by matrix instructions.

Arithmetic Intensity and Roofline Reasoning

Arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

The roofline model says:

$$P_{\text{achieved}} \leq \min(P_{\text{peak}}, B_{\text{memory}} I),$$

where P_{achieved} is achieved performance, P_{peak} is peak compute throughput, and B_{memory} is memory bandwidth.

A kernel is **memory-bound** when:

$$B_{\text{memory}} I < P_{\text{peak}}.$$

A kernel is **compute-bound** when:

$$B_{\text{memory}} I \geq P_{\text{peak}}.$$

For GEMM-like workloads, tiling increases reuse and raises arithmetic intensity. For memory-bound operations such as RMSNorm, dropout, residual addition, and RoPE, performance is often limited by bandwidth and memory coalescing rather than matrix core throughput.

Main Concepts

Why AMD Hardware Is Interesting

The lecture argues that AMD GPUs are not fundamentally weak hardware. On paper, recent AMD accelerators offer strong BF16 matrix core throughput, large HBM capacity, and high memory bandwidth. The issue is that the software stack is less mature, less uniform, and less forgiving for AI kernel development.

The AMD software ecosystem discussed includes:

- **PyTorch**: easy to use, but mostly optimized around common workloads and historically more mature on NVIDIA.
- **Triton**: productive tile-level programming, but the lecture claims that AMD performance can suffer due to compiler limitations.
- **Mojo, TileLang, CK, and similar systems**: useful abstractions, but not always proven as reusable primitives across AMD generations.
- **AOTriton and library kernels**: can be strong for specific cases, but coverage can be brittle.
- **Raw assembly**: maximum control, but poor scalability and high developer burden.

The central claim is:

AMD has strong hardware, but peak performance often requires hardware-aware code.

The Trade-off Between Abstraction and Performance

The lecture contrasts programming models by abstraction level:

thread-level $\rightarrow$ warp or wave-level $\rightarrow$ block-level $\rightarrow$ operator-level.

CUDA is thread-level. ThunderKittens and HipKittens are closer to the warp or wave level. Triton is closer to the block-tile level.

The trade-off is:

higher abstraction $\Rightarrow$ more productivity but more compiler responsibility,

lower abstraction $\Rightarrow$ more programmer control but more hardware burden.

At the block level, the compiler must infer how to map a block tile into waves, how to schedule producer and consumer work, how to allocate registers, how to lower memory accesses, and how to use hardware instructions. At the wave level, the programmer directly controls more of this structure.

Register Lifetime Tracking

Register lifetime tracking is the compiler problem of determining when each program value is live and assigning those values to physical registers. If the compiler fails to reuse registers effectively, the kernel may require too many registers. When register demand exceeds available physical registers, values spill to memory.

Let:

R_{needed} = number of registers needed by the compiler,

$R_{\text{available}}$ = number of physical registers available to the wave.

If:

$$R_{\text{needed}} > R_{\text{available}},$$

then the compiler may spill:

register value $\rightarrow$ scratch memory.

Spilling is expensive because values that should have lived in registers are moved through memory. For high-performance matrix kernels, register spills can destroy throughput.

Lowering Memory Accesses

Lowering memory accesses means transforming high-level loads and stores into the specific hardware instructions that best match the access pattern.

If each thread in a wave reads contiguous memory, the ideal hardware behavior is a coalesced transaction:

many per-thread loads $\rightarrow$ one or a few wide memory transactions.

For example, if 64 threads each load a neighboring BF16 value, the hardware may be able to issue a wide contiguous load. A compiler that does not recognize this pattern might generate less efficient instructions.

The key condition for coalescing is that consecutive lanes access consecutive addresses:

$$\text{addr}(t) = \text{base} + t \cdot s,$$

where t is the lane index and s is the element size in bytes. When s matches the element size and addresses are aligned, coalescing is usually favorable.

Why Raw Assembly Performs Well but Does Not Scale

Raw assembly gives the programmer direct control over:

- exact matrix core instructions,
- exact registers used,
- instruction ordering,
- memory instructions,
- shared memory access patterns,
- barriers and waits,
- cache-sensitive scheduling.

The downside is that each new workload, dtype, shape, architecture, and fused operation may require substantial expert work. Assembly is hard to read, hard to maintain, and hard to generalize.

HipKittens tries to occupy a middle position:

raw assembly performance intuition+C++ template abstractions+tile-level programming interface.

ThunderKittens Background

The ThunderKittens Philosophy

ThunderKittens was designed around the observation that many AI kernels operate over regular tiles of tensors. Instead of exposing every thread directly, it offers tile-level primitives that map cleanly onto GPU hardware.

The central abstraction is a **tile**. A tile can live in:

- global memory,
- shared memory,
- registers.

A kernel author writes operations over tiles:

load tile $\rightarrow$ matrix multiply tile $\rightarrow$ store tile.

ThunderKittens is C++ embedded, but intentionally has a Python-like style. This makes it more approachable to AI systems researchers while still letting expert programmers inspect or modify low-level implementations.

Producer-Consumer Scheduling in ThunderKittens

One major ThunderKittens abstraction is a producer-consumer template. The idea is:

producer warps $\rightarrow$ move data,

consumer warps $\rightarrow$ compute on data.

The producer loads tiles from HBM to shared memory. The consumer reads from shared memory into registers and issues matrix core instructions. The template handles much of the synchronization and pipelining.

A simplified schedule is:

time	producer	consumer	shared memory state
0	load tile 0	idle	filling
1	load tile 1	compute tile 0	double-buffered
2	load tile 2	compute tile 1	double-buffered
3	load tile 3	compute tile 2	double-buffered

The purpose is to overlap memory and compute:

$$T_{\text{pipeline}} \approx T_{\text{startup}} + N \cdot \max(T_{\text{load}}, T_{\text{compute}}) + T_{\text{drain}}.$$

Why HipKittens Cannot Merely Copy ThunderKittens

HipKittens keeps a similar high-level style, but the backend implementation must change because AMD differs from NVIDIA in several important ways:

- AMD waves are typically 64 threads rather than 32-thread warps.
- AMD matrix core layouts do not map cleanly to the same small core matrix abstraction.
- AMD register files and compiler behavior impose different constraints.
- Shared memory behavior has AMD-specific bank and phase behavior.
- AMD memory movement instructions have different semantics.
- AMD chiplet cache topology creates different cache reuse problems.
- NVIDIA-style producer-consumer scheduling does not transfer directly because AMD lacks some fine-grained asynchrony and register reallocation features.

HipKittens Design

Research Question

The main research question is:

Are entirely new programming primitives needed for AMD kernel development,
or can ThunderKittens-style primitives be adapted?

The answer from the lecture is nuanced. The high-level programming style can be preserved, but the implementation must be deeply AMD-specific. HipKittens therefore keeps recognizable tile abstractions while changing how tiles map to registers, shared memory, memory instructions, and schedules.

Three Optimization Axes

HipKittens organizes performance around three axes:

1. **Memory movement:** ensure data reaches matrix cores in the correct layout with coalesced global accesses and conflict-free shared memory accesses.
2. **Occupancy:** schedule waves so memory and compute resources are simultaneously utilized.
3. **Cache reuse:** order thread blocks so chiplet-local caches are reused effectively.

The total optimization goal can be written as:

$$\max P_{\text{kernel}} = f(U_{\text{matrix}}, U_{\text{memory}}, U_{\text{cache}}, U_{\text{occupancy}}),$$

where U_{matrix} is matrix core utilization, U_{memory} is memory bandwidth utilization, U_{cache} is cache hit effectiveness, and $U_{\text{occupancy}}$ is ability to hide latency and keep execution units active.

Hardware-Level Explanation

AMD Versus NVIDIA at a High Level

The lecture does not claim that AMD or NVIDIA is categorically better. Instead, it emphasizes that their designs differ enough that software cannot treat them as interchangeable.

Important NVIDIA features mentioned include:

- TMA, or Tensor Memory Accelerator, for asynchronous memory movement into shared memory.
- Asynchronous matrix multiply instructions in newer architectures.
- Fine-grained synchronization primitives such as memory barriers.
- Register reallocation mechanisms that help producer-consumer schedules.

Important AMD features mentioned include:

- 64-thread waves.
- Large register files.
- Smaller shared memory relative to some NVIDIA designs.
- Multiple cache levels, including chiplet-local L2 and shared last-level cache.
- Matrix core instructions with AMD-specific register layouts.
- Memory instructions that can move data into LDS, AMD's shared memory.

Registers and AMD Register Partitioning

The lecture highlights an important AMD issue: the register pool can be partitioned into accumulator general-purpose registers and vector general-purpose registers.

A simplified model is:

$$R_{\text{total}} = R_{\text{AGPR}} + R_{\text{VGPR}}.$$

For example, under one occupancy regime:

$$512 = 256 + 256.$$

The problem is that the compiler may not generate all desired cases where accumulator registers are used as inputs to matrix instructions. This can force inefficient movement or cause register pressure.

If the ideal kernel would use:

$$R_{\text{ideal}} = R_{\text{AGPR}} + R_{\text{VGPR}}$$

flexibly, but the compiler restricts how values flow between register classes, then the effective register capacity is smaller than the physical register capacity.

Explicit Register Pinning

HipKittens introduces explicit register pinning. The programmer or library can assign a register tile to a known register range.

Conceptually:

$$\text{register tile} \rightarrow \{r_i, r_{i+1}, \dots, r_j\}.$$

If a tile uses registers 78 through 93, then:

$$R_{\text{tile}} = \{r_{78}, r_{79}, \dots, r_{93}\}.$$

This can prevent the compiler from making poor allocation decisions. It is a way of recovering some of the control that raw assembly programmers have, while still writing in a higher-level library.

Matrix Core Register Layouts

Matrix core instructions require that input operands be placed in specific registers owned by specific lanes. The instruction does not accept an abstract matrix. It accepts a distributed register layout across a wave.

For a matrix multiply:

$$D_{m \times n} \leftarrow A_{m \times k} B_{k \times n} + C_{m \times n},$$

each lane owns a subset of elements from A , B , C , and D . The layout is hardware-defined.

On NVIDIA, ThunderKittens can use a relatively regular small core matrix abstraction. On AMD, the lecture explains that a simple repeated 16×16 style core matrix does not universally capture the matrix core layouts. Therefore, HipKittens register tiles must carry more information about the underlying core shape and instruction mapping.

A conceptual HipKittens tile type must encode:

$$\text{tile} = (\text{logical shape}, \text{dtype}, \text{memory space}, \text{core shape}, \text{matrix instruction}).$$

The important point is that the programmer still calls an MMA-like primitive, but the tile definition must contain enough structure to map correctly to AMD matrix core instructions.

Shared Memory and LDS

AMD shared memory is often referred to as LDS, or local data share. Shared memory is banked. Bank conflicts occur when multiple lanes in the same access phase try to access addresses mapping to the same bank.

If the bank index is:

$$b(a) = \left\lfloor \frac{a}{w} \right\rfloor \bmod B,$$

where a is an address, w is the bank width, and B is the number of banks, then two accesses conflict when:

$$b(a_i) = b(a_j) \quad \text{for } i \neq j$$

within the same phase.

The lecture describes reverse-engineered behavior for AMD LDS read instructions. Some instructions appear to expose 64-bank behavior in specific phases, while other instructions behave like 32-bank accesses. This matters because a layout that is conflict-free for one instruction may conflict for another.

Phased Shared Memory Instructions

For an instruction such as a 128-bit LDS read, the lecture describes access phases. Not all lanes participate in the same way at the same moment. Instead, the hardware processes groups of lanes across phases.

A simplified model is:

$$\text{instruction} = \text{phase}_0 + \text{phase}_1 + \text{phase}_2 + \text{phase}_3.$$

A bank conflict is only relevant among lanes active in the same phase. If phase p has active lane set S_p , then the conflict condition is:

$$\exists i, j \in S_p, i \neq j \quad \text{such that} \quad b(a_i) = b(a_j).$$

This is why reverse engineering phase structure matters. It determines which lane pairs can conflict.

Reverse Engineering Shared Memory Behavior

The lecture describes a practical method for reverse engineering undocumented shared memory behavior:

1. Mask out all but one or two lanes.

2. Force selected lanes to access chosen LDS addresses.
3. Measure whether bank conflicts occur.
4. Sweep lane pairs to infer phase membership.
5. Sweep address offsets to infer whether the behavior is 32-bank or 64-bank.

This is a microbenchmarking approach. It treats the hardware as an observable system and infers hidden rules from timing and conflict metrics.

Visual Concept

Draw a 64-lane AMD wave split into four sequential phases. Under each phase, show a subset of active lanes connected to a banked LDS array. Highlight two lanes hitting the same bank to show a conflict.

Global Memory to LDS Movement

AMD provides memory instructions that can move data from global memory into LDS. The lecture compares this to NVIDIA TMA, but notes that the details differ.

A key difference is where swizzling occurs. On NVIDIA, one common model is to swizzle the shared-memory layout. On AMD, the lecture explains that the effective swizzle may be achieved by swizzling the global memory addresses used when loading into LDS.

Conceptually, each lane writes to a structured LDS location:

$$\text{lds_addr}(t) = g(t),$$

but the global source address is chosen as:

$$\text{global_addr}(t) = f(t),$$

where f is designed so that the final LDS arrangement is bank-conflict-free for later reads.

Mathematical Reasoning

Tiled GEMM Work Decomposition

For GEMM:

$$D = AB + C,$$

where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C, D \in \mathbb{R}^{M \times N}.$$

A thread block computes one output tile:

$$D_{i:i+B_M, j:j+B_N}.$$

The block loops over K in chunks of size B_K :

$$D_{ij} = \sum_{q=0}^{K/B_K-1} A_{i:i+B_M, qB_K:(q+1)B_K} B_{qB_K:(q+1)B_K, j:j+B_N}.$$

The total arithmetic for one output tile is:

$$\text{FLOPs}_{\text{tile}} = 2B_M B_N K.$$

If each K -tile loads:

$$B_M B_K + B_K B_N$$

elements, and each element has s bytes, then approximate bytes loaded per output tile are:

$$\text{Bytes}_{\text{tile}} \approx s (B_M K + K B_N) + s B_M B_N.$$

The arithmetic intensity is approximately:

$$I_{\text{tile}} \approx \frac{2B_M B_N K}{s (B_M K + K B_N + B_M B_N)}.$$

For large K , this becomes:

$$I_{\text{tile}} \approx \frac{2B_M B_N}{s (B_M + B_N)}.$$

This shows why larger tiles improve reuse. Increasing B_M and B_N increases arithmetic per byte moved.

Occupancy and Register Pressure

Occupancy depends on how many waves can reside on a CU. A simplified occupancy bound is:

$$W_{\text{resident}} \leq \min \left(W_{\text{max}}, \left\lfloor \frac{R_{\text{CU}}}{R_{\text{wave}}} \right\rfloor, \left\lfloor \frac{S_{\text{CU}}}{S_{\text{block}}} \right\rfloor W_{\text{block}}, B_{\text{limit}} W_{\text{block}} \right),$$

where R_{CU} is registers per CU, R_{wave} is registers per wave, S_{CU} is shared memory per CU, S_{block} is shared memory per block, W_{block} is waves per block, and B_{limit} is the maximum resident block count.

If register pressure increases, then:

$$R_{\text{wave}} \uparrow \Rightarrow W_{\text{resident}} \downarrow.$$

Lower resident waves can reduce latency hiding and reduce overlap between memory and compute.

Producer-Consumer Register Imbalance

In a producer-consumer kernel, producer waves move data and consumer waves compute. Producers usually need fewer registers:

$$R_{\text{producer}} < R_{\text{consumer}}.$$

On NVIDIA, register reallocation can help allocate more registers to consumers and fewer to producers. If the hardware or compiler does not support this efficiently, the allocation may become uniform:

$$R_{\text{assigned, producer}} = R_{\text{assigned, consumer}}.$$

This wastes registers on producers:

$$R_{\text{wasted}} = R_{\text{assigned, producer}} - R_{\text{needed, producer}}.$$

The cost is that consumers receive fewer effective resources, which may force smaller tiles or fewer accumulators.

Ping-Pong Scheduling Model

The lecture describes an AMD-friendly ping-pong schedule. Instead of strict NVIDIA-style producer-consumer warp specialization, two waves per SIMD alternate between bulk memory and bulk compute.

For two waves W_0 and W_1 , the schedule is:

Step	W_0	W_1
0	load tile 0	wait
1	compute tile 0	load tile 1
2	load tile 2	compute tile 1
3	compute tile 2	load tile 3

The goal is:

$$T_{\text{step}} \approx \max(T_{\text{load}}, T_{\text{compute}}),$$

rather than:

$$T_{\text{step, sequential}} = T_{\text{load}} + T_{\text{compute}}.$$

The speedup from overlap is approximately:

$$S = \frac{T_{\text{load}} + T_{\text{compute}}}{\max(T_{\text{load}}, T_{\text{compute}})}.$$

When load and compute times are balanced:

$$T_{\text{load}} \approx T_{\text{compute}},$$

the ideal overlap speedup approaches:

$$S \approx 2.$$

Chiplet Cache Reuse Model

On AMD chiplet GPUs, work can be distributed across multiple XCDs. Each XCD has private L2 cache, and the full GPU has shared lower-level memory resources. The lecture emphasizes that naive block ordering can cause blocks assigned to the same XCD to access unrelated tiles, losing cache reuse.

Let a block compute output tile:

$$D_{i,j}.$$

It loads:

$$A_{i,:} \quad \text{and} \quad B_{:,j}.$$

Two blocks $D_{i,j}$ and $D_{i,j+1}$ reuse the same $A_{i,:}$ tile. Two blocks $D_{i,j}$ and $D_{i+1,j}$ reuse the same $B_{:,j}$ tile.

A cache-aware scheduler tries to assign nearby blocks to the same XCD within a time window so that:

$$\text{reuse distance} < \text{cache residency window}.$$

The lecture describes two tuning parameters:

- C : chunk size, or number of consecutive block IDs assigned to an XCD.
- W : window size, controlling the two-dimensional shape of the block assignment pattern.

The goal is to choose C and W to increase L2 reuse:

$$\max_{C,W} H_{L2}(C,W),$$

where H_{L2} is the L2 hit rate or effective cache reuse metric.

Code Walkthrough

Representative HipKittens Tile-Level GEMM

The transcript describes HipKittens as a C++ header-style library with ThunderKittens-like tile primitives. The following is a representative sketch, not an exact source listing. Its purpose is to show the programming model.

```
template<int BM, int BN, int BK>
__global__ void hk_gemm_kernel(
    const bf16* __restrict__ A,
    const bf16* __restrict__ B,
    bf16* __restrict__ D,
    int M, int N, int K) {

    int block_m = blockIdx.y;
    int block_n = blockIdx.x;

    // Register accumulator tile.
    hk::rtile<bf16, BM, BN, hk::core_shape::mfma_compatible> acc;
```

```

// Shared memory tiles.
__shared__ bf16 smem_a[BM * BK];
__shared__ bf16 smem_b[BK * BN];

hk::zero(acc);

for (int k0 = 0; k0 < K; k0 += BK) {
    hk::load_global_to_shared_swizzled(
        smem_a,
        A + block_m * BM * K + k0,
        M, K);

    hk::load_global_to_shared_swizzled(
        smem_b,
        B + k0 * N + block_n * BN,
        K, N);

    __syncthreads();

    hk::rtil<bf16, BM, BK, hk::core_shape::mfma_compatible> a_reg;
    hk::rtil<bf16, BK, BN, hk::core_shape::mfma_compatible> b_reg;

    hk::load_shared_to_register(a_reg, smem_a);
    hk::load_shared_to_register(b_reg, smem_b);

    hk::mma(acc, a_reg, b_reg, acc);

    __syncthreads();
}

hk::store_register_to_global(
    D + block_m * BM * N + block_n * BN,
    acc,
    M, N);
}

```

Explanation of the GEMM Sketch

The line:

```

int block_m = blockIdx.y;
int block_n = blockIdx.x;

```

maps each thread block to one output tile:

$$D_{block_m, block_n}.$$

The register accumulator:

```
hk::rtil<bf16, BM, BN, hk::core_shape::mfma_compatible> acc;
```

represents an output tile in registers. The additional core-shape information is important on AMD because matrix core instructions impose a specific distributed register layout across lanes.

The shared memory allocations:

```
__shared__ bf16 smem_a[BM * BK];  
__shared__ bf16 smem_b[BK * BN];
```

stage A and B tiles. In an optimized implementation, these arrays are not merely flat arrays. Their layouts are chosen or swizzled to avoid LDS bank conflicts.

The loop over K :

```
for (int k0 = 0; k0 < K; k0 += BK)
```

implements the tiled GEMM accumulation:

$$D_{m,n} = \sum_{k_0} A_{m,k_0:k_0+B_K} B_{k_0:k_0+B_K,n}.$$

The global-to-shared load:

```
hk::load_global_to_shared_swizzled(...)
```

must do several hardware-sensitive things:

- ensure global memory accesses are coalesced,
- align accesses to hardware transaction boundaries,
- swizzle source addresses if needed so LDS layout is conflict-free,
- use efficient AMD memory instructions when available.

The shared-to-register load:

```
hk::load_shared_to_register(a_reg, smem_a);
```

must match the MFMA instruction's expected operand layout. This is not a generic matrix load. It distributes elements across wave lanes and registers in the exact pattern expected by the matrix core.

The MMA call:

```
hk::mma(acc, a_reg, b_reg, acc);
```

maps to AMD matrix core instructions. The hardware consumes the distributed register fragments and updates accumulator registers.

Representative Explicit Register Pinning

The lecture discusses explicit register pinning as a way to avoid compiler register allocation problems. The following is representative pseudocode.

```
using acc_regs = hk::pinned_register_range<78, 93>;

hk::rtile<
    bf16,
    BM,
    BN,
    hk::core_shape::mfma_compatible,
    acc_regs
> d_tile;
```

This expresses:

$$d_tile \rightarrow \{r_{78}, r_{79}, \dots, r_{93}\}.$$

The hardware implication is that the programmer prevents the compiler from freely moving the accumulator tile into an inefficient register allocation. This is especially valuable when the compiler has limitations around AGPR and VGPR usage.

Representative Ping-Pong Schedule

A simplified ping-pong loop can be sketched as follows.

```
for (int stage = 0; stage < num_stages; ++stage) {
    if ((wave_id + stage) % 2 == 0) {
        hk::bulk_load_next_tile(stage);
        hk::conditional_barrier_arrive(stage);
    } else {
        hk::conditional_barrier_wait(stage);
        hk::compute_current_tile(stage);
    }
}
```

The key idea is staggered wave progress. One wave is advanced into memory work while another wave computes. A conditional barrier creates a persistent offset between waves.

At a high level:

$$W_0 : \text{load} \rightarrow \text{compute} \rightarrow \text{load} \rightarrow \text{compute},$$

$$W_1 : \text{wait} \rightarrow \text{load} \rightarrow \text{compute} \rightarrow \text{load}.$$

This schedule preserves larger tile-level programming while still creating memory-compute overlap on AMD.

Representative Cache-Aware Block Reordering

The lecture describes tuning the grid order with chunk size C and window size W . The following pseudocode shows the idea.

```
int logical_id = blockIdx.x;
int xcd_id = logical_id % NUM_XCDS;

int chunk = logical_id / (NUM_XCDS * C);
int within = (logical_id / NUM_XCDS) % C;

int window_row = chunk / W;
int window_col = chunk % W;

int tile_m = window_row * C + within;
int tile_n = window_col;
```

This is not the exact algorithm from the paper; it is a conceptual sketch. The point is that block IDs are remapped so that blocks assigned near each other in time and to the same XCD share either A -tile or B -tile data.

The hardware goal is:

same XCD + nearby work + overlapping operand tiles $\Rightarrow$ higher L2 reuse.

Performance Intuition

Why Tensor Core Utilization Dominates

Recent GPU generations have increased matrix core throughput much faster than scalar or vector throughput. Therefore, a high-performance AI kernel must keep matrix cores fed.

The performance target is:

$$U_{\text{matrix}} = \frac{\text{active matrix core cycles}}{\text{total cycles}} \approx 1.$$

Low matrix utilization may be caused by:

- waiting for global memory,
- shared memory bank conflicts,
- register spills,

- poor operand layout,
- insufficient tile reuse,
- synchronization overhead,
- low occupancy,
- poor cache locality.

Why Memory Layout Is Not Cosmetic

A matrix tile is not just a two-dimensional mathematical object. It is distributed across lanes, registers, shared memory banks, cache lines, and HBM transactions.

The same logical tile can have very different performance depending on layout:

logical tile $\neq$ physical layout.

The physical layout determines:

- whether global loads are coalesced,
- whether LDS reads conflict,
- whether matrix core instructions accept the operands directly,
- whether extra shuffles or register moves are needed.

Why Compiler Limitations Matter More Near the Hardware

At high abstraction levels, a compiler can hide many details. But in peak-performance kernels, the developer is often relying on exact instruction sequences. Small compiler mistakes can cause large performance losses.

Examples from the lecture include:

- failing to reclaim registers,
- spilling registers to scratch memory,
- failing to lower vectorized loads to the best memory instructions,
- failing to generate desired AGPR input cases for matrix instructions.

A useful diagnostic is:

unexpected scratch memory traffic $\Rightarrow$ possible register spilling.

Another diagnostic is:

low memory transaction efficiency $\Rightarrow$ possible poor load lowering or poor coalescing.

Why NVIDIA Producer-Consumer Does Not Transfer Directly

Producer-consumer scheduling works well on NVIDIA partly because the hardware and programming model provide strong support for:

- asynchronous memory movement,
- explicit synchronization primitives,
- register reallocation,
- warp specialization.

On AMD, the synchronization overhead for GEMM-like workloads may be manageable, but lack of equivalent register reallocation is a major issue. Producer waves may reserve registers they do not need, reducing resources available to compute-heavy consumer waves.

Therefore, HipKittens explores alternatives:

- four-way finely interleaved schedules, similar to hand-written assembly;
- eight-wave ping-pong schedules, which preserve tile-level structure while overlapping memory and compute.

Four-Way Fine Interleaving

The raw assembly approach often maps one wave to one SIMD and manually interleaves memory and compute instructions.

A simplified stream is:

load $\rightarrow$ mfma $\rightarrow$ load $\rightarrow$ mfma $\rightarrow$ store $\rightarrow$ mfma.

This can achieve excellent occupancy and latency hiding because the programmer directly controls instruction order. The cost is that it is hard to write and maintain.

Eight-Wave Ping-Pong

The ping-pong schedule uses two waves per SIMD and alternates their roles. It can be thought of as a middle ground:

more structured than raw assembly,
more AMD-friendly than NVIDIA-style producer-consumer.

This is especially useful when the workload permits bulk memory operations and bulk compute operations to alternate cleanly.

Why Cache Reuse Becomes More Important on Chiplet GPUs

If a GPU has multiple chiplets with private L2 caches, then block scheduling affects which data remains local.

In naive row-major scheduling, two nearby block IDs may be sent round-robin to different XCDs. This can scatter reuse across private caches.

The effective cost of a tile load becomes:

$$\text{cost} = P_{\text{hit}}C_{\text{L2}} + (1 - P_{\text{hit}})C_{\text{HBM}},$$

where P_{hit} is L2 hit probability. Since:

$$C_{\text{HBM}} \gg C_{\text{L2}},$$

even modest improvements in P_{hit} can improve performance.

AMD-Specific Kernel Engineering Details

Wave Size and Tile Granularity

A 64-thread wave changes how tile fragments are distributed. Compared with a 32-thread warp, a 64-thread wave has more lanes participating in a single instruction. This affects:

- register fragment layout,
- LDS phase behavior,
- memory coalescing width,
- matrix instruction operand mapping,
- occupancy calculations.

A tile abstraction must be aware of this. A tile shape that is natural for NVIDIA may not be natural for AMD.

Core Matrix Shape Complexity

HipKittens requires the programmer or template system to specify core shape because AMD matrix instructions do not always decompose into a single elegant repeated core matrix.

The relationship is:

$$\text{logical tile shape} + \text{core shape} + \text{instruction} \rightarrow \text{valid register layout}.$$

If the core shape is wrong, the matrix core instruction may consume the wrong values or require expensive rearrangement.

Bank Conflicts in LDS

Shared memory bank conflicts serialize accesses. If n lanes in the same phase hit the same bank, the access may be serialized roughly by a factor related to n :

$$T_{\text{conflict}} \approx nT_{\text{single}}$$

for severe conflicts, though exact behavior depends on hardware.

The practical goal is:

$$\forall p, \quad b(a_i) \neq b(a_j) \quad \text{for most active lane pairs } i, j \in S_p.$$

HipKittens hides many common conflict-free patterns inside optimized load functions.

Global Coalescing

For global memory, the ideal is that lanes access contiguous, aligned addresses.

If a wave loads elements of size s , then good coalescing often resembles:

$$a_t = a_0 + ts.$$

Poor coalescing resembles:

$$a_t = a_0 + \pi(t)s,$$

where $\pi(t)$ is an irregular permutation or stride that causes scattered memory transactions.

In GEMM, coalescing can be complicated by the need to transform data into LDS layouts that are later conflict-free. AMD may require swizzling global source addresses while writing to structured LDS destinations.

Memory-Bound Kernels

The lecture mentions kernels such as fused dropout-residual-normalization and RoPE. These are typically memory-bound. For such kernels, matrix core optimization is irrelevant; the main concerns are:

- reduce number of global memory passes,
- fuse operations,

- use vectorized loads and stores,
- avoid unnecessary intermediate tensors,
- ensure coalesced accesses,
- keep register pressure reasonable.

For a fused residual and RMSNorm-like operation:

$$y_i = \frac{x_i + r_i}{\sqrt{\frac{1}{d} \sum_{j=1}^d (x_j + r_j)^2 + \epsilon}} \gamma_i.$$

The arithmetic per element is modest, but each element may require multiple memory reads and writes. Fusing avoids writing intermediate $x + r$ to HBM.

Code Logic for Common AI Kernels

Attention Forward

A FlashAttention-style forward pass computes:

$$O = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V.$$

The naive implementation materializes:

$$S = QK^\top,$$

where:

$$S \in \mathbb{R}^{L \times L}.$$

FlashAttention avoids materializing S by tiling over K and V , maintaining online softmax statistics.

For a row block, define:

$$m_i = \max_j s_{ij},$$

$$\ell_i = \sum_j \exp(s_{ij} - m_i),$$

$$o_i = \sum_j \exp(s_{ij} - m_i) v_j.$$

When processing a new tile, update:

$$m_i^{\text{new}} = \max(m_i^{\text{old}}, m_i^{\text{tile}}),$$

$$\ell_i^{\text{new}} = \exp(m_i^{\text{old}} - m_i^{\text{new}}) \ell_i^{\text{old}} + \exp(m_i^{\text{tile}} - m_i^{\text{new}}) \ell_i^{\text{tile}},$$

$$o_i^{\text{new}} = \exp(m_i^{\text{old}} - m_i^{\text{new}}) o_i^{\text{old}} + \exp(m_i^{\text{tile}} - m_i^{\text{new}}) o_i^{\text{tile}}.$$

The output is:

$$O_i = \frac{o_i}{\ell_i}.$$

On AMD, the hard parts are:

- mapping $QK^\top$ and PV matrix multiplies to MFMA instructions,
- keeping Q , K , and V tiles in correct register and LDS layouts,
- avoiding bank conflicts in shared memory,
- handling softmax vector operations without starving matrix cores,
- managing register pressure from accumulators and online softmax state.

Grouped Query Attention Backward

The transcript mentions difficult workloads such as GQA causal backward. These kernels are challenging because they combine:

- matrix multiplies,
- causal masking,
- reductions,
- softmax derivative logic,
- multiple tensor layouts,
- high register pressure,
- irregular reuse patterns.

The softmax backward identity is:

$$dS_{ij} = P_{ij} \left(dP_{ij} - \sum_k P_{ik} dP_{ik} \right).$$

For attention:

$$P = \text{softmax}(S), \quad O = PV.$$

Then:

$$dV = P^\top dO,$$

$$dP = dOV^\top,$$

$$dQ = dSK,$$

$$dK = dS^\top Q.$$

The backward kernel is harder than forward because it must coordinate more matrix products and reductions while respecting causal masks and avoiding extra HBM traffic.

Comparison With Other Programming Systems

PyTorch

PyTorch is excellent for productivity:

few lines of Python $\rightarrow$ working model.

But PyTorch operators may not immediately support every new architecture, dtype, fusion pattern, or model architecture. If a new architecture requires custom kernels, waiting for PyTorch library support may be too slow.

PyTorch is best when:

- the operation is common,
- the backend implementation is mature,
- portability matters more than peak performance,
- kernel fusion needs are modest.

Triton

Triton is a block-level tile language. The developer writes code over program instances and block tensors. It is highly productive, but the compiler must perform many lower-level decisions.

The lecture argues that, on AMD, Triton can struggle with:

- register lifetime tracking,
- register spills,
- lowering memory accesses to optimal intrinsics,
- exploiting all AMD-specific memory and matrix instructions,
- generating schedules comparable to hand-tuned kernels.

The deeper point is not that block-level programming is impossible, but that the compiler must become extremely hardware-aware.

Mojo, TileLang, and Closed Compiler Systems

The lecture treats closed-source compiler systems pragmatically: if they produce fast kernels, then their compiler figured out the necessary transformations. If benchmarks are weak, it is difficult to know why without seeing the compiler.

The key evaluation criterion is empirical:

programming model quality = productivity+achieved performance+portability across hardware generations.

Composable Kernel and Assembly Libraries

AMD's Composable Kernel and internal assembly-based libraries can provide high performance for supported cases. However, coverage and extensibility are difficult. If an operation falls outside the hand-tuned path, it may fall back to a slower backend.

This creates a long-tail problem:

many AI workloads $\neq$ small set of hand-written kernels.

HipKittens aims to make the long tail easier to cover.

Common Misconceptions

Misconception: Bigger Hardware Numbers Automatically Mean Faster Kernels

A GPU may have high peak BF16 throughput and memory bandwidth, but kernels only achieve that performance if software maps correctly to the hardware.

$$P_{\text{actual}} \ll P_{\text{peak}}$$

when memory movement, layouts, scheduling, or compiler code generation are poor.

Misconception: AMD Is Just NVIDIA With Different Names

AMD and NVIDIA are both GPUs, but their detailed execution models differ. AMD has waves rather than warps, different matrix core layouts, different shared memory behavior, different cache topology, and different compiler constraints.

The correct view is:

$$\text{same broad GPU model} \neq \text{same kernel optimization strategy}.$$

Misconception: Tile Abstractions Are Always Hardware-Agnostic

Tiles are a useful programming abstraction, but tile layout is hardware-specific. A tile must ultimately become registers, LDS addresses, global memory transactions, and matrix instructions.

$$\text{tile abstraction} \rightarrow \text{hardware-specific lowering}.$$

Misconception: Producer-Consumer Scheduling Is Universally Optimal

Producer-consumer is powerful on NVIDIA, but it depends on hardware support. On AMD, the register allocation and reallocation story differs, so alternative schedules may be better.

Misconception: Shared Memory Is Always Faster

Shared memory is only faster when accessed well. Bank conflicts can serialize LDS operations and reduce performance. A bad shared memory layout can be worse than expected.

Misconception: Compiler Problems Are Minor Details

For peak kernels, compiler behavior is central. Register spills, missed vectorization, and poor intrinsic selection can dominate runtime.

Practical Takeaways

For GPU Kernel Engineers

1. Start from the hardware execution model, not from abstract tensor algebra.
2. Identify where each tile lives: HBM, cache, LDS, or registers.
3. Confirm that register layouts match matrix core instructions.
4. Measure register spills and scratch memory traffic.
5. Inspect whether global accesses are coalesced.
6. Check LDS bank conflicts with profiling or microbenchmarks.
7. Think carefully about wave scheduling and occupancy.
8. For chiplet GPUs, treat block ordering as a cache optimization problem.

For AMD Kernel Development

The most important AMD-specific lessons are:

- Wave-level programming is valuable because many optimizations are below the block level.
- Explicit register control may be needed to bypass compiler limitations.
- LDS behavior can be instruction-specific and must be measured.
- Global-to-LDS memory movement may require address swizzling.
- NVIDIA-style producer-consumer should not be copied blindly.
- Cache-aware scheduling matters because XCD-local L2 caches change reuse behavior.

For DSL Designers

A GPU DSL must decide where it sits in the abstraction stack.

higher-level DSL $\Rightarrow$ compiler must discover more hardware details,

lower-level DSL $\Rightarrow$ developer must express more hardware details.

HipKittens chooses a lower-level, wave-aware design because AMD performance requires many hardware-specific choices that are hard to infer automatically.

For Deep Learning Systems Researchers

A theoretically efficient architecture is not enough. A model architecture is only practically efficient if it maps well to hardware.

For example, a model with fewer asymptotic operations may still be slower if it fails to use matrix cores:

lower algorithmic complexity $\nrightarrow$ lower wall-clock time.

The correct question is:

Does this architecture expose large, regular, reusable tiles that feed matrix cores?

Project or Experiment Ideas

Benchmark Naive Versus Tiled GEMM on AMD

Implement:

1. naive GEMM,
2. shared-memory tiled GEMM,
3. MFMA-based tiled GEMM,
4. HipKittens-style tiled GEMM.

Measure:

$$\text{TFLOPs} = \frac{2MNK}{T \cdot 10^{12}}.$$

Compare against theoretical peak.

Measure LDS Bank Conflicts

Write a microbenchmark that varies:

- lane ID,
- LDS address stride,
- instruction width,
- active lane mask.

Infer:

$$B \in \{32, 64\}$$

for different LDS instructions by observing when conflicts occur.

Register Pinning Experiment

Implement the same small MFMA kernel with and without explicit register pinning. Measure:

- register count,
- spills,
- scratch memory traffic,
- achieved throughput,
- occupancy.

The hypothesis is:

$$\text{pinning} \Rightarrow \text{fewer spills} \Rightarrow \text{higher throughput.}$$

Ping-Pong Versus Sequential Scheduling

Implement a tiled GEMM loop in two schedules:

$$\text{sequential} = \text{load} \rightarrow \text{compute} \rightarrow \text{load} \rightarrow \text{compute},$$

$$\text{ping-pong} = \text{overlap load and compute across waves.}$$

Measure whether overlap improves performance and under what tile sizes.

Cache-Aware Grid Ordering

For GEMM, implement different block orderings:

- row-major,
- column-major,
- grouped ordering,
- XCD-aware chunk and window ordering.

Measure L2 hit rate and kernel time. Sweep C and W :

$$(C^*, W^*) = \arg \max_{C, W} P_{\text{kernel}}(C, W).$$

Memory-Bound Kernel Fusion

Implement fused dropout, residual addition, and RMSNorm:

$$y_i = \frac{\text{dropout}(x_i) + r_i}{\sqrt{\frac{1}{d} \sum_j (\text{dropout}(x_j) + r_j)^2 + \epsilon}} \gamma_i.$$

Compare against separate PyTorch operators. Measure HBM bytes moved and effective bandwidth.

Hardware Profiling Checklist

When optimizing an AMD AI kernel, collect the following information:

1. **Matrix core utilization:** Are MFMA instructions active most cycles?
2. **Global memory throughput:** Is the kernel saturating HBM?
3. **LDS bank conflicts:** Are shared memory accesses serialized?
4. **Register spills:** Is scratch memory traffic unexpectedly high?
5. **Occupancy:** How many waves are resident per CU?
6. **Cache hit rates:** Are XCD-local L2 caches reused effectively?
7. **Instruction mix:** Is time spent on matrix instructions, vector ALU, memory instructions, or waits?
8. **Synchronization stalls:** Are barriers or waits dominating?

A useful diagnostic equation is:

$$T_{\text{kernel}} = T_{\text{mfma}} + T_{\text{mem}} + T_{\text{lds}} + T_{\text{spill}} + T_{\text{sync}} + T_{\text{other}}.$$

Optimization consists of identifying which term is unexpectedly large.

Final Mental Model

HipKittens is best understood as a hardware-near tile programming system for AMD GPUs. It keeps the productivity idea of ThunderKittens but replaces the backend assumptions with AMD-specific knowledge.

The lecture's final mental model is:

AI kernels are tile programs,

but:

tiles only run fast when their physical layout matches the GPU.

On AMD, matching the GPU means:

- distributing register fragments according to MFMA expectations,
- controlling register allocation when the compiler is insufficient,
- avoiding LDS bank conflicts using instruction-aware layouts,
- coalescing and swizzling global memory movement,
- choosing schedules that fit AMD's wave and register model,
- reordering blocks to exploit XCD-local cache reuse.

The essence of HipKittens is not merely porting ThunderKittens to AMD. It is the process of discovering which low-level hardware facts must be exposed, which can be hidden, and which abstractions are still productive after the hardware differences are respected.

Peak AMD performance = matrix cores fed + registers controlled + LDS conflicts avoided + HBM coalesced -
--